Knowledge Distillation and Deployment of a Lightweight Command-Parsing Language Model on a Local Cloud

Student: Anubhav Kumar Thakur | **Roll Number:** 230101002
Course: CS3081 – Cloud Computing

Task 1 — Problem Definition, Architecture and Design

1.1 Objective

The objective of this project was to design, train, evaluate and deploy a lightweight command-parsing language model as a locally hosted cloud service. The system converts natural-language commands into structured JSON containing fields such as intent, date, time and place.

The project combines **knowledge distillation, lightweight language-model fine-tuning, object storage, containerization, Kubernetes orchestration, networking, reliability testing and CI/CD** into a single reproducible local-cloud deployment.

The teacher model specified for the project was **Phi-3.5-mini-instruct (3.8B)**. A smaller **EleutherAI Pythia-70M** model was trained using responses generated by the teacher. The objective was not to reproduce the teacher's full language capability, but to specialize the smaller model for the command-parsing task.

1.2 Overall Architecture

The final system flows from **command pool → teacher model → distillation dataset → Pythia-70M student → evaluation → model packaging → SeaweedFS S3 → FastAPI → Docker → Kubernetes → Ingress → client** (Figure 1). The training pipeline uses 3,000 command inputs from the supplied command pool. Teacher-generated responses were collected and cleaned, with a maximum of 1,500 examples used for student training.

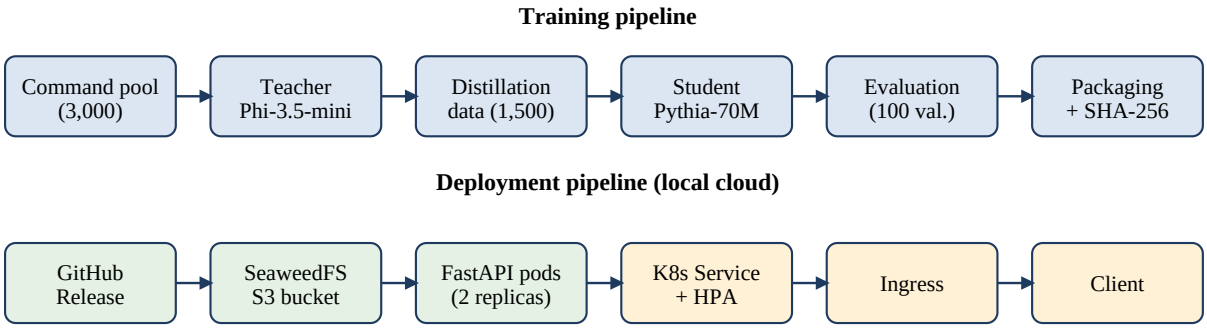


Figure 1: End-to-end architecture of the training and deployment pipeline.

The deployment architecture contains:

- **FastAPI** inference service and the **Pythia-70M** lightweight student model.
- **SeaweedFS** providing S3-compatible object storage; a **Docker** container for the inference service.
- **k3d/Kubernetes** for orchestration, with a **Service and Ingress** for request routing and an **HPA** for CPU-based scaling.
- **GitHub Actions** for linting, testing and model evaluation; **GitHub Release** for versioned model artifacts.

The model is deliberately **not embedded inside the Docker image**. At application startup, the model is downloaded from object storage. This separates the application image from model artifacts and allows model versions to be changed without rebuilding the complete application image.

1.3 Deployment Interface

The FastAPI application exposes the endpoints below. Responses from `/parse` also carry the headers `X-Model-Version` and `X-Served-By`, which were used during reliability and load-balancing experiments to determine which model version and Kubernetes replica served each request.

Endpoint	Purpose
`POST /parse`	Parse a natural-language command into structured JSON
`GET /healthz`	Liveness check
`GET /readyz`	Readiness check

1.4 Design Decisions

Several design decisions were made to satisfy the local-cloud constraint. First, Kubernetes was implemented using **k3d**, avoiding dependence on an external cloud provider. Second, **SeaweedFS** was selected as local S3-compatible object storage. Third, the student model was kept lightweight so that inference could be performed on the local cluster. The Kubernetes Deployment initially uses two replicas, and CPU and memory requests/limits are configured so that resource consumption is controlled and the HPA can make scaling decisions.

Measured result: a fresh deployment successfully downloaded the model from SeaweedFS, loaded it into both application replicas, and exposed the `/parse` endpoint through the Ingress.

Task 2 — Knowledge Distillation, Training and Model Evaluation

2.1 Teacher–Student Training

The teacher model was **Phi-3.5-mini-instruct** and the student was **EleutherAI/Pythia-70M**. The supplied command pool contained **3,000 commands**. Teacher responses were generated using temperature 0 and seed 42 as specified by the assignment, stored exactly, and used as the source for student training. The first **1,500 cleaned records** were used for training, satisfying the assignment limit.

The student was trained with a causal language-model objective. During development, an initial configuration produced excessive EOS/repetitive outputs. This was addressed by using a full-sequence causal-language-model objective, training a fresh model and using a more conservative learning rate. The final training configuration is given in Table 1.

Table 1: Final training configuration.

Parameter	Value
Student model	EleutherAI/Pythia-70M
Training examples	1,500
Maximum sequence length	256
Batch size	8
Epochs	3
Learning rate	2×10^{-5}
Weight decay	0.01
Gradient clipping	1.0

Measured fact: training was performed on a Kaggle GPU because CPU training on the local machine was impractically slow. This should not be interpreted as evidence that the deployment requires a GPU; the deployed inference service runs on CPU resources.

2.2 Evaluation Dataset

The supplied validation dataset contains **100 labeled commands**. Predictions were generated using the trained student model and scored with the supplied `score.py`, which measures JSON validity, intent accuracy, date accuracy, time accuracy, place accuracy and full-frame accuracy.

2.3 Measured Evaluation Results

Table 2: Student-model results on the 100-example validation set.

Metric	Result
JSON validity	**99%**
Intent accuracy	**83%**
Date accuracy	**76%**
Time accuracy	**82%**
Place accuracy	**81%**
Full-frame accuracy	**44%**
Validation examples	**100**

The required CI quality gate was `json_validity ≥ 95%`; the measured validity of **99%** passes it. The lower full-frame score shows that producing syntactically valid JSON is easier than correctly predicting every semantic field simultaneously — an important distinction when evaluating command-parsing models.

2.4 Interpretation

The 70M student learned a useful subset of the command-parsing behavior from the teacher-generated dataset. However, the **44% full-frame accuracy** indicates that the model is not yet sufficiently reliable for applications where every field must be correct. The main improvement opportunities are additional high-quality training examples, better handling of structured outputs and constrained decoding.

Task 3 — Model Packaging, Versioning and Object Storage

The trained model was packaged independently from the application as `pythia-70m-command-parser.zip` and published as a **GitHub Release asset** under the `final` release rather than being committed into Git. The archive SHA-256 checksum, also recorded in `models/v1/model.sha256`, is:

```
1539f339dd6f056a7a934fa45abd1294b7e486f292096f75d0f8bd6d6cd2d3c6
```

The deployment process downloads the release model and uploads the required model files to the local SeaweedFS S3 bucket, with the following layout:

```
model-storage/pythia-70m-command-parser/
    config.json, generation_config.json, model.safetensors,
    model_metadata.json, SHA256SUMS.txt, smoke_test_results.json,
    tokenizer.json, tokenizer_config.json
```

3.1 Why Object Storage Was Used

Separating the model from the container image provides three advantages: the application image remains smaller; different model versions can be stored independently; and Kubernetes can download the appropriate model during startup based on environment variables (`MODEL_URI`, `MODEL_VERSION`, `S3_ENDPOINT`, `MAX_NEW_TOKENS`). Credentials are supplied through a Kubernetes Secret rather than being hard-coded.

3.2 Measured Deployment Verification

A clean deployment successfully (1) downloaded the release model, (2) verified its SHA-256 checksum, (3) uploaded eight model files to SeaweedFS, (4) started the FastAPI application, (5) downloaded the model from S3 during startup, (6) loaded it successfully and (7) passed the readiness check. This verifies that deployment does not depend on an untracked local model

directory.

Task 4 — Containerization, Kubernetes Deployment and Reliability

4.1 Containerization

The FastAPI service was packaged using Docker; the Kubernetes deployment uses the image `command-parser:s3-v1`, exposing port **8000**. Resource settings are in Table 3. Liveness and readiness probes use `/healthz` and `/readyz`. The Deployment initially runs **two replicas**; the HPA uses min 1, max 2 replicas and a 70% CPU target. A rolling-update strategy with controlled surge/unavailability reduces interruption during model-version changes (Figure 2).

Table 3: Pod resource requests and limits.

Resource	Request	Limit
CPU	500m	2 CPU
Memory	1 GiB	2 GiB

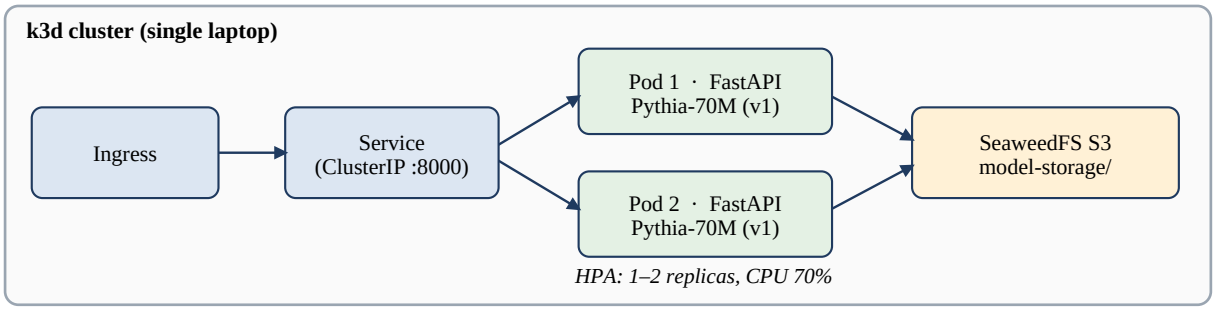


Figure 2: Kubernetes deployment topology on the local k3d cluster.

4.2 Load Balancing

With two Ready replicas, the standard load test generated **157 requests**; results are in Table 4. Successful requests were distributed as Replica 1: **80 (51.28%)** and Replica 2: **76 (48.72%)**, so both replicas served well above the required 30% share.

Table 4: Standard load-test results (two replicas).

Metric	Result
Requests / successful / failed	157 / 156 / 1
Error rate	**0.64%**
p50 latency	**2.4 s**
p95 latency	**6.4 s**
Throughput	**1.32 req/s**

4.3 Pod Recovery

One running application Pod was deleted while the service was under load. The experiment produced **156 requests** with **1 failed request (0.71% error rate)**. The Deployment automatically recreated the Pod and restored the replica count. The error rate was below the required **2%** recovery threshold.

4.4 Cold Start

All application Pods were deleted and recreated. The three measured creation-to-Ready times were **13 s, 12 s and 13 s**, giving a **median cold-start time of 13 seconds**. Application logs confirmed that the model files were downloaded from S3 before the service became ready.

4.5 Rolling Update and Rollback

A model-version change from **v1 to v2** was performed under load: 299 requests, 3 failures (**1.00% error rate**), p50 = 3.3 s, p95 = 5.9 s. The required rolling-update error threshold was **≤0.5%**, so this

experiment **failed the specified criterion**. Investigation showed the failures were associated with invalid/repetitive output from the v2 model rather than a Kubernetes routing failure.

The deployment was then rolled back with `kubectl rollout undo`, and the service again reported `X-Model-Version: v1`, demonstrating successful rollback. The result also shows why model validation should occur before production promotion.

Task 5 — Network Design and Security

The network was designed in the required address space `10.20.0.0/16` with four logical subnets (Table 5).

Table 5: Subnet plan.

Subnet	CIDR	Usable addresses
App	10.20.0.0/22	1022
Public	10.20.4.0/24	254
Data	10.20.5.0/25	126
Management	10.20.5.128/27	30

The public-subnet Network ACL uses ordered, stateless rules. The first inbound rule denies traffic from `198.51.100.0/24`. TCP ports 80 and 443 are allowed from anywhere, and SSH on port 22 is restricted to the management subnet. Replies from the application subnet are explicitly permitted because the NACL is stateless. Outbound rules allow application traffic to TCP port 8000 and ephemeral reply traffic to clients. Rule order matters: the deny rule is evaluated before the broader allow rules.

5.1 NACL Verification

The supplied packet test suite was run with `python tools/nacl_sim.py network/nacl.json network/packets_sample.jsonl --check`. **Measured result: 6/6 packets passed (100%)**, confirming the rules match the supplied packet expectations.

5.2 Security Decisions

Credentials were not placed in application source code; SeaweedFS access credentials are injected through Kubernetes Secrets. The model artifact is checksum-verified before deployment, reducing the possibility of silently deploying a corrupted or unexpected archive.

Task 6 — Reproducibility, Testing and CI/CD

The project provides a reproducible deployment path through the Makefile. `make up` creates or reuses the local k3d cluster, installs/configures SeaweedFS, downloads and verifies the release model, uploads it to object storage, builds/imports the application image and deploys the Kubernetes resources. `make down` deletes the local cluster.

6.1 Automated Testing

Unit tests cover request/response schema validation, health and readiness endpoints, and parsing behavior. **Measured result: 9 tests passed**, and static checking with Ruff completed successfully.

6.2 CI Pipeline

The GitHub Actions pipeline runs in the required order: **(1) linting, (2) unit tests, (3) model evaluation, (4) secret scan**. The model-evaluation stage downloads the model from the GitHub Release instead of relying on a model committed into the repository, and runs the supplied scoring script with the quality gate `--min-validity 95`. The final valid-model CI run passed with the measured **99% JSON validity**.

6.3 Deliberate Failure

A deliberately invalid model artifact was used to demonstrate that the pipeline does not blindly accept an invalid model. The failed GitHub Actions run was preserved as evidence in the

repository/report; the valid model was then restored and CI passed.

6.4 Secret Scanning and AI Usage

Gitleaks is included as a CI stage for detecting accidentally committed secrets. An `AI_USE.md` file documents the use of AI assistance (explanations, debugging, code suggestions and documentation); implementation, testing, model training, deployment experiments and verification were performed and checked by the student.

Task 7 — Performance, Cost, SLO and Discussion

7.1 Performance

The main load-balancing experiment (Table 4) is the performance reference. The assignment target requires error rate ≤1% and p95 ≤5 s. The measured error rate of **0.64% satisfies** the error-rate target, but the measured **6.4 s p95 exceeds** the 5 s target, so the performance requirement is only **partially satisfied**. The latency is influenced by CPU-based inference of a language model and the local Kubernetes environment, and should not be read as the theoretical maximum performance of the model or FastAPI service.

7.2 Resource Utilization

During one observed deployment, Kubernetes reported approximately **2m CPU** and **445 MiB** memory. These are point-in-time observations and are **not used for the cost calculation**, because the supplied rate card bills on configured CPU and memory limits.

7.3 Cost Estimate

The supplied rate card specifies **₹3.50 per vCPU-hour** and **₹0.45 per GiB-hour**. Each replica has a 2 vCPU and 2 GiB limit, so the assumed cost per replica is (2 × 3.50) + (2 × 0.45) = **₹7.90/hour**, and ₹15.80/hour for two replicas. At the measured throughput, 1.32 × 3600 = 4,752 requests/hour, hence:

$$\textbf{Cost per 1,000 requests} = (15.80 / 4{,}752) \times 1{,}000 \approx ₹3.33$$

Assumption: ₹3.33 per 1,000 requests is an exercise estimate based on the supplied hypothetical rate card and measured throughput. It is not a real cloud-provider price.

7.4 SLO and Error Budget

A 99% availability SLO corresponds to a 1% error budget. The main load test measured 0.64% errors, leaving 1.00 − 0.64 = **0.36%** of budget; it therefore consumed approximately **64%** of the error budget. The rolling-update experiment produced 1.00% errors, consuming the entire 1% budget for that experiment, and also exceeded the specific 0.5% rolling-update requirement.

Limitations and Improvements

Limitations. (1) The Kubernetes cluster runs on a single laptop, so it provides no true multi-machine fault tolerance: Pod recovery was demonstrated, but failure of the underlying machine would affect the whole system. (2) The student's **44% full-frame accuracy** shows semantic parsing remains a major limitation; valid JSON does not guarantee correct fields. (3) The measured p95 latency of **6.4 s** exceeds the 5 s target. (4) The v2 rolling update produced a **1.00% failure rate**, exceeding the 0.5% threshold, showing that model promotion needs stronger validation.

Future improvements.

- Increase the distillation dataset with more diverse commands.
- Use constrained JSON decoding to improve structured-output reliability.
- Quantize the student model to reduce inference latency and memory consumption.
- Automate validation of a candidate model before Kubernetes promotion, and introduce canary deployment for model versions.

- Use a more capable student model if resources allow, and optimize token generation and maximum output length.
- Deploy across multiple physical nodes for genuine fault tolerance.

Conclusion

This project implemented an end-to-end local-cloud workflow for training and deploying a lightweight command-parsing language model. A **70M-parameter Pythia student** trained from **1,500 teacher-generated examples** achieved **99% JSON validity, 83% intent accuracy and 44% full-frame accuracy** on the supplied validation set. The model was packaged as a versioned, checksum-verified artifact, stored in SeaweedFS and loaded dynamically by a FastAPI application running in Kubernetes.

The infrastructure experiments demonstrated load balancing (51.28% / 48.72% across two replicas), Pod recovery (0.71% errors), a median cold start of 13 s and successful rollback. The NACL simulator passed 6/6 tests, and CI performed linting, unit testing, model evaluation and secret scanning, passing the 99% validity gate.

The project also identified important limitations: p95 latency of **6.4 s** (target 5 s) and **1.00%** errors in the v2 rolling update (limit 0.5%). These failures show that successful deployment requires not only infrastructure reliability but also validation of the model itself before promotion. Overall, the project demonstrates a complete, reproducible local-cloud ML deployment pipeline while clearly identifying the remaining gaps in model quality, latency and safe model-version promotion.

References

[1] Microsoft, *Phi-3.5-mini-instruct*, Hugging Face model documentation. [2] EleutherAI, *Pythia Model Suite*, Hugging Face. [3] Kubernetes Documentation: Deployments, Services, Probes and Horizontal Pod Autoscaling. [4] SeaweedFS Documentation, S3-compatible object storage. [5] FastAPI Documentation. [6] Docker Documentation. [7] GitHub Actions Documentation. [8] Locust Documentation. [9] Course-provided CS3081 assignment specification and student kit.

Appendix A — Key Configuration Values

Item	Value	Item	Value
Student model	EleutherAI/Pythia-70M	Initial replicas	2
Training examples	1500	HPA	min=1, max=2, CPU=70%
Validation examples	100	Model version	v1
API	FastAPI :8000	Maximum new tokens	128
Object storage	SeaweedFS S3	Kubernetes	k3d

Appendix B — Final Measured Results

Experiment	Result	Requirement	Status
JSON validity	99%	≥95%	PASS
Load-test error rate	0.64%	≤1%	PASS
Load-test p95	6.4 s	≤5 s	FAIL
Replica distribution	51.28 / 48.72%	≥30% each	PASS
Pod recovery	0.71% errors	≤2%	PASS
Cold-start median	13 s	Measure/report	PASS
Rolling update	1.00% errors	≤0.5%	FAIL
Rollback	v1 restored	v1	PASS
NACL tests	6/6	6/6	PASS
Unit tests	9/9	≥3	PASS
Model quality gate	99% validity	≥95%	PASS
Estimated cost	₹3.33 / 1000 req.	Calculate	—